

[What's new?](#) | [Help](#) | [Directory](#) | [Sign in](#)

mostpixelsever

Multi-Screen Library for Processing

Search projects

[Project Home](#) [Downloads](#) [Wiki](#) [Issues](#) [Source](#)Search for

ProcessingTutorial

Tutorial on how to use the MPE library with the Processing IDE.

Updated Sep 26, 2008 by

[daniel.shiffman](#)Before you try this tutorial, you probably want to read [HowThisWorks](#).Labels: [Featured](#), [Tutorial](#)

Step 1. Run the server.

Download and extract [MPE server application](#)

Run the server via the command line.

```
java -jar mpeServer.jar
```

```
Terminal — java — 92x8
[daniel] $ java -jar mpeServer.jar
WallServer: framerate = 30, screens = 2, master dimensions = 640, 240
Starting server: daniel-shiffmans-powerbook-g4-15.local/192.168.1.109 9002
[]
```

The server can be configured by editing the mpeServer.ini file. Comments are made with "#". Parameters are set like so "name=value;" No spaces! End with a semi-colon!

```
#"screens" variable contains the total number of screens
screens=2;

#"framerate" variable contains the global framerate of the screens
framerate=30;
```

Step 2. Download and install the MPE library

Grab the library from the [download page](#)

3. Convert a single-screen applet to a multi-screen one!

Let's say you have a [simple ball bouncing applet](#) that runs at 320x240. We now want to make it run at 640x240 across two windows. (For testing, it's easiest to run two windows on one machine.)

First, import the library:

```
import mpe.client.*;
```

Then create a client object:

```
// A client object
Client client;
```

We also need to include a boolean variable to keep track of whether we've started or not. This is a bit of a hack and exists because we want to disable Processing's automatic looping (so that all clients execute draw() in sync) and Processing always runs through draw at least one time. The boolean allows us to skip that first time.

```
boolean start = false;
```

In setup(), initialize the Client object. Just as with the server, a settings INI file is required. This file tells the client what its local dimensions are, as well as its location within the master dimensions. Here is an example:

```
# Client ID (0,1,2,etc.)
```

```

id=0;
# Server address (localhost for testing)
server=localhost;
# Server port
port=9002;
# What are the client window dimensions?
localScreenSize=200,400;
# Where is the client located in the master dimensions?
localLocation=0,0;
# What are the master dimensions?
masterDimensions=400,400;
# Turn to 0 to turn off all the console printing
debug=1;

```

Once you've create the INI file, you can make the client object in setup():

```

// Make a new Client with an INI file.
// sketchPath() is used so that the INI file is local to the sketch
client = new Client(sketchPath("mpe.ini"),this);

```

Once you've made the Client object, you should ask it for the size of your sketch.

```

// The size is determined by the client's local width and height
size(client.getLWidth(), client.getLHeight());

```

Finally, in setup(), you must make sure to disable Processing's draw() loop by calling noLoop(), as well as tell the client to start().

```

// Important, must start the client!
client.start();
// Crucial, must stop the automatic looping of Processing!
noLoop();

```

Processing's draw() loop is replaced by the the frameEvent() callback. This function is triggered by the Client object whenever the server alerts the client that a new frame should be rendered. You must implement this function for the framework to function properly.

// This is triggered by the client whenever a new frame should be rendered void frameEvent(Client c){

```

    start = true; // Yes, we've really started now! redraw();

```

```

}

```

Ok, almost there! Now, whenever you run through draw() you must (a) make sure we've really started and (b) ask the client to place itself within the larger display. Finally, you must (c) client.done() to tell the server you've finished.

```

public void draw() {
    if (start) {
        // Before we do anything, the client must place itself within the larger display
        // (This is done with translate, so use push/pop if you want to overlay any info on all screens)
        client.placeScreen();

        // Do whatever it is you would normally do
        background(255);
        for (int i = 0; i < balls.size(); i++) {
            Ball ball = (Ball) balls.get(i);
            ball.calc();
            ball.display();
        }

        // Alert the server that you've finished drawing a frame
        client.done();
    }
}

```

Note that if you are using the variables width or height in your sketch, you may need to change these to client.getMWidth();

```

if (x < 0 || x > client.getMWidth()) xdir *= -1; // Note the use of the master width!

```

In addition, if you are using any random numbers or Perlin noise, you must make sure to seed them with a constant so that all client screens will pick the same sequence of randoms.

```

randomSeed(1); // Seed random so all clients pick same sequence
noiseSeed(1);

```

Broadcasting Messages

You can also broadcast a message from one client to all clients. For example, if you want a mouse connected to one client to act as a mouse for the entire system, you will need to broadcast a message when the mouse is clicked.

```
void mousePressed() {
  // How to broadcast a message
  // Do not include a ":" in your message
  int x = mouseX + client.getXOffset();
  int y = mouseY + client.getYOffset();
  client.broadcast(x + "," + y);
}
```

Note the use of `getXOffset()` and `getYOffset()` to account for the mouse's position within the master dimensions.

The message is then received during a `frameEvent()`. Note that the message comes in as an array of Strings (the server accumulates these messages into an array.)

```
void frameEvent(Client c){
  if (c.messageAvailable()) {
    String[] msg = c.getDataMessage();
    // Only using the first message in the array
    // We could loop through them all here just in case
    int[] xy = int(msg[0].split(","));
    balls.add(new Ball(xy[0],xy[1]));
  }
  redraw();
}
```

Step 4. Ok, how the heck do I run this thing?

Going back to the bouncing ball example, we want to have two windows 320x240 each, together making a bigger window 640x240. The server should have `masterDimensions=640x240` and each client will have local dimensions of 320x240.

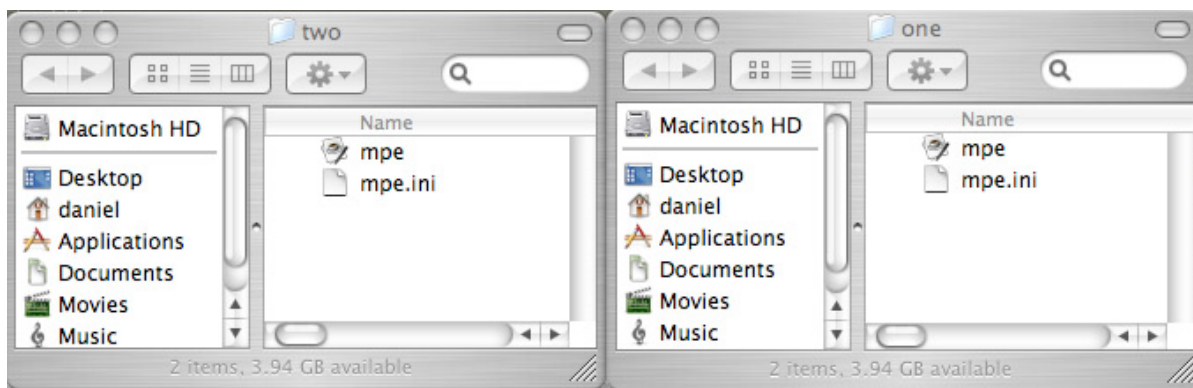
You should now go ahead and export your sketch to application, and make a copy of that application. You should then have two applications and two INI files. These would ultimately live on separate computers, but for now you can place them in separate folders. Note the application is identical, only the INI file is different!

One INI file would look like this:

```
id=0;
localLocation=0,0;
server=localhost;
port=9002;
localScreenSize=320,240;
masterDimensions=640,240;
```

And the other like this:

```
id=1;
localLocation=320,0;
server=localhost;
port=9002;
localScreenSize=320,240;
masterDimensions=640,240;
```



You can download the example sketch from the downloads page:

<http://code.google.com/p/mostpixelsever/downloads/list>

Comment by [zimmen](#), Sep 30, 2008

What i'd like to know is how you managed to get a huge video running in processing! The default library isn't up to hd footage.

Comment by [xbilidade](#), Oct 01, 2008

I thing that code: // A client object Client client;

Should be: // A client object UDPClient client;

update the tutorial

Comment by [s.benoit.studio](#), Nov 30, 2008

hi! I've got a problem on step 1: Where exactly have we to extract the server? My terminal doesn't find the .jar file when i run the command line; or doesn't permit me to launch it when i give an absolute path. :-(

Got an idea? -

Comment by [traumasystems](#), Mar 04, 2009

Anybody got any idea how to run the server on a windows machine i've downloaded Java and instaled it but what is the next step

Comment by [nabais.fernando](#), Apr 12, 2009

Hello all

Great work and thanks for making it available. I'm experimenting with it, I have a prototype running but would like to be able to generalize it by accessing the masterdimensions from within the applet. The examples use `client.getLWidth()`, etc, but I can find any reference to other public methods (like `masterDimensions`, for example). Is there a reference somewhere?

Thanks

Comment by [nabais.fernando](#), Apr 14, 2009

sorry, it's when I post a question that I become clever :-) there's the L, for local and M for Master (who would say ;-)

► [Sign in](#) to add a comment

©2009 Google - [Code Home](#) - [Terms of Service](#) - [Privacy Policy](#) - [Site Directory](#)

Hosted by 